MÓDULO 2 · ACTIVIDAD M2 · RETO 2

Alerta temprana para una flota vehicular

1000 eventos en 30 s, un `Emergency` y un correo a Gmail. Solución desplegada en **AWS serverless**; la misma arquitectura corre en **local con Docker** para desarrollo y como plan B de la demo.

Group 2 · Daniel Mazo · Freddy Aparicio · Camilo Céspedes
Diplomado en Arquitectura de Software y Cloud Computing · PUJ Cali · octubre 2026

01 · EL PROBLEMA

Lo que pide el enunciado, en números

CARGA

1000

eventos en 30 s con el k6 oficial (10 VUs). El 100 % debe procesarse.

LÍMITE DE TASA

15 r/s

en el API Gateway (rate). Burst 2000 en la imagen del enunciado.

PROCESADORES

≤ 10

instancias activas a la vez (Lambda, contenedores, ECS…).

CORREO

< 15 s

para 2,5 puntos. 15–45 s → 1,5 · > 45 s → 0,5.

La tensión de fondo. 1000 peticiones en 30 s son unas 33–36 r/s, más del doble del rate de 15. Lo que hace posible el «100 % procesado» es el **burst de 2000**, no el rate.

Dos criterios de tiempo. El PDF dice «del último envío de k6 al envío y recepción del correo», con corte en 15 s. En el encuentro del 06/10, el docente habló de **< 16 s desde el envío del `Emergency` hasta que el correo llega**. Medimos los dos: hasta el envío (logs) y hasta la llegada al buzón (cabeceras de Gmail).

Fuente: enunciado de la Actividad Reto 2 · criterio de 16 s: encuentro sincrónico del 06/10

02 · ATRIBUTO DE CALIDAD PRIORIZADO

Priorizamos desempeño y escalabilidad

Priorizamos que la alerta llegue rápido (**latencia** del `Emergency` al correo) y que el sistema **absorba la ráfaga** de 1000 eventos escalando procesadores, porque es lo que decide si la alerta sirve: a un conductor en emergencia le importa que alguien se entere rápido, **a costa de** que un `200` confirme «guardado» y no «enviado», y de una pieza más (la cola).

DESEMPEÑO

Latencia de la alerta

El correo sale en paralelo a la recepción: la petición no espera el envío. Medido en AWS: correo en el buzón en ≤ 1,35 s contra 15 s.

ESCALABILIDAD

Absorber la ráfaga

Lambda abre instancias según la carga dentro de un cupo reservado; la cola desacopla el ritmo de entrada del de envío.

RESTRICCIÓN DURA

No perder un Emergency

Si no se puede encolar, se responde 503. Entre perder y duplicar, elegimos duplicar: un correo repetido molesta, una emergencia perdida no se recupera.

A COSTA DE

Asincronía

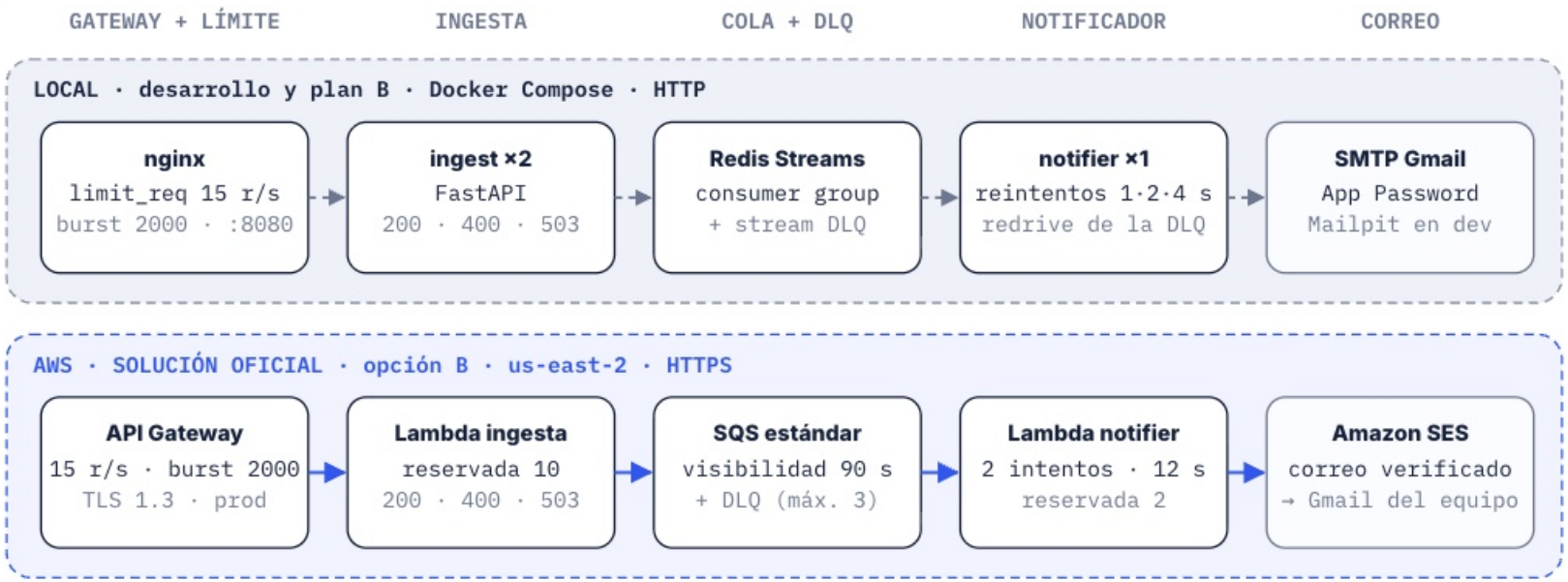
El `200` confirma que la alerta quedó guardada, no que el correo salió. Y hay una pieza más que operar: la cola.

Fuente: Documento técnico (Entregable 2), §2 · README.md del código fuente (Entregable 1)

03 · DIAGRAMA DE ARQUITECTURA

Una arquitectura lógica: oficial en AWS, desarrollo en local

Gateway con límite → ingesta que responde ya → cola → notificador asíncrono → correo. Lo que cambia entre entornos son los **adaptadores**, no el patrón.



Entra: k6 (10 VUs, 1000 iteraciones). **Position** termina en la ingesta con 200. **Emergency** se registra, se encola y responde 200; el correo sale después, fuera de la petición. Logs JSON con hora Colombia en cada paso (archivos locales / CloudWatch).

04 · SOLUCIÓN OFICIAL: AWS

AWS opción B: serverless en us-east-2

Cinco servicios gestionados en una región elegida midiendo: con k6 en 4 regiones, us-east-2 ganó en promedio.



1 Responder sin esperar el correo

La ingesta responde **200** en milisegundos y encola solo los Emergency: el correo no ocupa el camino que mide k6.

2 SQS desacopla y no pierde

Cola durable + DLQ: si el notifier falla o está ocupado, la alerta espera. Entrega **al menos una vez**.

3 Correo acotado en tiempo

2 intentos de ≤ 5 s y 1 s de espera: **≤ 12 s** antes de la DLQ. Corrida oficial: **0,756 s** de recepción a `EMAIL_SENT`.

Por qué serverless aquí ✓ Escala automática ✓ Sin servidores que administrar ✓ HTTPS incluido en API Gateway ✓ Pago por uso

- Configurado a mano en la consola (sin IaC) · se usaron 8 instancias, con un cupo que permitiría 12 (límite: 10).

Fuente: Documento técnico (Entregable 2), §9.4 y §10 · aws/README.md (Entregable 1). Diseño y despliegue de Camilo Céspedes; corrida oficial del 13/10 en README.md de los logs (Entregable 3).

05 · TÁCTICAS DE ARQUITECTURA

Cada táctica, en un componente concreto

TÁCTICA (DECK DEL CURSO)	DÓNDE, EN AWS	CÓMO AYUDA AL RETO	ATRIBUTO
Gestionar la llegada de eventos · limitar la tasa	API Gateway, stage `prod`: 15 r/s, burst 2000	Absorbe la ráfaga de 1000 sin rechazos y protege la ingesta	Desempeño
Limitar la respuesta al evento · encolar	SQS, solo para el Emergency	La ingesta responde 200 en milisegundos; el correo sale aparte	Desempeño
Tiempos de ejecución acotados	Timeout de SES 2 + 3 s; Lambda 15 s	Un correo lento no se come la banda de 15 s	Desempeño
Introducir concurrencia	Instancias de Lambda en paralelo	1000 peticiones en ~22 s con 6 instancias de ingesta	Escalabilidad
Programar recursos · cupo fijo	Concurrencia reservada por Lambda	Acota cuántas instancias se abren como máximo; pico real: 6 de ingesta + 2 del notifier = 8	Escalabilidad
Retry con límite	Notifier: 2 intentos → DLQ	Un fallo pasajero de SES no pierde la alerta	No perder
Detección y manejo de excepciones	Si SQS falla → 503, nunca 200	Quien envía sabe que debe reintentar	No perder
Reintroducción · reclamar pendientes	Visibilidad de SQS (90 s) y redrive (máx. 3)	Si el notifier muere, el mensaje vuelve a la cola	No perder

Nombres según el material del curso sobre tácticas de desempeño (Control Resource Demand / Manage Resources) y de disponibilidad (Exception detection, Retry). La fila de reintroducción es interpretación del equipo. Ver Documento técnico (Entregable 2), §3.

06 · DECISIONES DE ARQUITECTURA

Cuatro decisiones, cada una con su costo

DECISIÓN	ATRIBUTO	POR QUÉ	A COSTA DE
Responder 200 y encolar solo el Emergency	Desempeño	El correo no frena la petición: k6 recibe respuesta en milisegundos	El `200` confirma «guardado», no «enviado»; una pieza más (la cola)
Lambda con concurrencia reservada	Escalabilidad	Abre instancias según la carga, dentro de un cupo dimensionado para el peor escenario (10 VUs de k6)	Usamos **8** instancias (6 + 2), pero el cupo configurado permite hasta 12
SQS + DLQ con reintentos acotados (≤ 12 s)	Restricción: no perder	Si el correo falla, la alerta espera en la cola sin salirse de los 15 s	Se rinde antes: más casos van a la DLQ. **Brecha:** si SES no responde, el SDK reintenta por su cuenta y el peor caso llega a ≈23 s
Sin idempotencia	Restricción: no perder	Entre perder y duplicar, duplicar es lo menos grave; el enunciado no la exige y el `event_id` permitiría deduplicar	Correo duplicado posible

Fuente: Documento técnico (Entregable 2), §4 · local/README.md y aws/README.md (Entregable 1)

07 · CONTRATO HTTP

Qué garantiza nuestro `200 OK`, y cuándo aparece el `429`

`200`

Válido y guardado

Position: validado y registrado. Emergency: **ya encolado**. No garantiza que el correo salió ni que llegó.

`503`

No se pudo encolar

Preferimos un error visible a confirmar una alerta perdida. Quien envía debe reintentar.

`429`

Se agotó el burst

Lo responde el gateway (nginx o API Gateway) antes de llegar a la ingesta.

Prueba del limitador (local, 04/10, burst bajado a 100 a propósito)

MODO	200	429	ESPERADO POR TOKEN BUCKET
Ráfaga	102	898	≈ 100 del cubo + lo recargado en < 1 s
Ritmo de referencia (~35 r/s)	522	478	`100 + 15 × 28 = 520`
Burst 2000 (configuración real)	**1000**	**0**	**1000 < 2000: el cubo nunca se vacía**

Abierto: en el encuentro del 06/10 se pidió un `200` por cada una de las 1000 peticiones. Con burst 2000 se cumple, pero por encima de ese volumen el sistema respondería `429`, en los dos entornos. Cambiarlo (aceptar todo y encolar) movería el límite de 15 r/s del gateway al consumo de la cola.

Fuente: Documento técnico (Entregable 2), §5 · local/README.md, «Prueba del limitador» (Entregable 1)

08 · EVIDENCIA · CORRIDA OFICIAL

Del k6 del profesor al buzón de Gmail en ≤ 1,35 s

AWS, 13/10, k6 oficial: **1000/1000** aceptadas, 0 rechazadas (429), 1 `Emergency`, 1 correo al primer intento, 0 en la DLQ.



HORA (COLOMBIA)	EVENTO	FUENTE
19:01:54.543	`EMERGENCY_RECEIVED` · placa RJJ-055 (petición 992 de 1000)	CloudWatch · ingesta
19:01:54.654	Última petición de k6 recibida	CloudWatch · ingesta
19:01:54.807	`EMERGENCY_ENQUEUED` · la ingesta responde 200	CloudWatch · ingesta
19:01:55.298	`EMAIL_SENT` · SES acepta el correo, intento 1	CloudWatch · notifier
19:01:55	**Llega al buzón de Gmail («Entregado tras 0 segundos»)**	**Cabecera `Received`**

Fuente: Documento técnico (Entregable 2), §12.2 · corrida-oficial-aws-2026-10-13.log y .csv, cabeceras y captura del correo en README.md (Entregable 3). Otras corridas con el mismo script, todas < 15 s: AWS 11/10 (1,642 s), 12/10 (1,352 s) y 13/10 #7 (1,256 s, 2 Emergency) · local 06/10 (1,33 s).

09 · DISPONIBILIDAD

Qué pasa si se cae un componente

SE CAE	MIENTRAS ESTÁ CAÍDO	AL VOLVER
Correo (SMTP / SES)	Local: reintentos 1·2·4 s y luego DLQ. AWS: 2 intentos y luego DLQ; ≤ 12 s si SES rechaza el correo, hasta ≈23 s si no responde (reintentos ocultos del SDK, brecha declarada).	Local: cada 15 s devuelve a la cola lo que lleva menos de 10 min en la DLQ (`EMAIL_REDRIVEN`). AWS: queda en la DLQ para revisión.
Cola (Redis / SQS)	El Emergency responde **503** y no se guarda. Los Position siguen con 200.	El notifier se reconecta solo. Lo que recibió 503 no se recupera: debe reenviarlo el cliente.
Notifier	Los Emergency se acumulan en la cola.	Local: reclama los pendientes (`PENDING_CLAIMED`). AWS: SQS los vuelve a ofrecer pasada la visibilidad (90 s).

Documento técnico (Entregable 2), §13 · Local: verificado el 11/10 apagando cada componente desde el panel (local/README.md, Entregable 1). AWS: caso EMAIL_RETRY → EMAIL_DLQ en 2,05 s verificado; la caída a mitad de envío es comportamiento documentado de SQS, no forzado.

10 · PLANO DE CONTROL

Un panel para observar, con un switch Local / AWS

QUÉ ES

Una página que lanza el k6 del profesor, envía un Emergency manual y muestra la medida de la rúbrica con semáforo, el historial de corridas y la descarga de logs (.log / .csv). **No está en el camino de los eventos**: observar no altera la latencia medida.

SWITCH LOCAL / AWS

Un interruptor cambia la fuente sin tocar el resto. **Local** lee los logs, Redis y Docker. **AWS** lee CloudWatch Logs y el estado de SQS con un usuario IAM de **solo lectura**, y apunta k6 y el Emergency manual al API Gateway. En AWS no hay inyección de fallas: el panel no tiene permisos para apagar nada.

Lección del M1, aplicada: lo que observa no puede estar en el camino de lo que se mide. Por eso el panel corre aparte y no cuenta como procesador.

Fuente: Documento técnico (Entregable 2), §8 · local/README.md, «Panel de control» (Entregable 1). Switch verificado contra AWS el 12/10; los logs oficiales se exportaron desde este panel.

CIERRE

Priorizamos desempeño y escalabilidad, sin perder un Emergency: antes duplicar una alerta que perderla

Gateway con límite, ingesta que responde ya, cola, notificador asíncrono. Desplegada en AWS serverless: 1000/1000 y el correo en el buzón en ≤ 1,35 s contra un corte de 15 s.

Brechas declaradas: reparto 10 + 2 frente a ≤ 10 · reintentos ocultos del SDK (peor caso ≈23 s) · configuración de AWS sin IaC · DMARC sin dominio propio.